

CS202: Computer Organization - Lecture Notes

Lecture 2: Between Your Program and Hardware

Levels of Program Code

- **High-level language:**
 - Level of abstraction closer to problem domain.
 - Provides for productivity and portability.
- **Assembly language:**
 - Textual representation of instructions.
- **Hardware representation:**
 - Binary digits (bits).
 - Encoded instructions and data.

From High-Level to Hardware

- **High-level language program (in C):**

C

```
swap(int v[], int k) {  
    int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

- **Compiler -> Assembly language program (for MIPS):**

Code snippet

```
swap:  
    muli $2, $5, 4  
    add $2, $4, $2  
    lw $15, 0($2)  
    lw $16, 4($2)  
    sw $16, 0($2)  
    sw $15, 4($2)  
    jr $31
```

- **Assembler -> Binary machine language program (for MIPS):**

```
000000001010000100000000000011000  
0000000000001100000001100000100001  
...
```

Inside the Processor (CPU)

- Datapath: Performs operations on data.
- Control: Sequences datapath, memory, ...
- Cache Memory: Small fast SRAM memory for immediate access to data.

Abstractions

- **The ISA (Instruction Set Architecture):** The hardware/software interface.
- **ABI (Application Binary Interface):** The user/system interface.

Performance

- Define performance: Response time vs. Throughput.
- **Response time (Execution time):** How long it takes to do a task.
- **Throughput (Bandwidth):** Total work done per unit time.
- **Relative Performance:**
 - $\text{Performance}_x / \text{Performance}_y = \text{Execution_time}_y / \text{Execution_time}_x = n$
- **Measuring Execution Time:**
 - Elapsed time (Wall clock time): Total response time, includes all aspects (processing, I/O, OS overhead, idle time).
 - CPU time: Time spent processing a given job.
- **CPU Clocking:**
 - Clock period: duration of a clock cycle.
 - Clock frequency (rate): cycles per second.
 - $\text{CPU Time} = \text{CPU Clock Cycles} * \text{Clock Cycle Time} = \text{CPU Clock Cycles} / \text{Clock Rate}$.

Power Wall

- $\text{Power} = \text{Capacitive load} * \text{Voltage}^2 * \text{Frequency}$
- The "Power Wall": We cannot just increase clock frequency anymore because of thermal limits.

Amdahl's Law

- The performance enhancement possible with a given improvement is limited by the amount that the improved feature is used.
- $\text{Execution Time}_{\text{new}} = \text{Execution Time}_{\text{old_unaffected}} + (\text{Execution Time}_{\text{old_affected}} / \text{Amount of Improvement})$
- **Corollary:** Make the common case fast.

Lecture 3: Instruction Set Architecture

Outline

- Instruction set architecture
- RISC vs. CISC
- MIPS/ARM/x86
- **Instructions:**
 - Arithmetic instruction: add, sub, ...
 - Data transfer instruction: lw, sw, lh, sh, ...
 - Logical instruction: and, or, ...

- Conditional branch: beq, bne, ...
- **Basic concepts:**
 - Operands: register vs. memory vs. immediate
 - Numeric representation: signed, unsigned, sign extension
 - Instruction format: R-format vs. I-format

Instruction Set

- To command a computer's hardware, you must speak its language.
- **Instruction set:** vocabulary of commands.
- **Design Principle 1: Simplicity favors regularity.**
 - Keep all instructions a single size (e.g., 32 bits for MIPS).
 - Always 3 operands for arithmetic operations.
- **Design Principle 2: Smaller is faster.**
 - MIPS has 32 registers (*s0*—*s7*, *t0*—*t9*, etc.).

MIPS Arithmetic

- `add a, b, c` # $a = b + c$
- `sub a, b, c` # $a = b - c$
- C code: `f = (g + h) - (i + j);`
- MIPS code:

Code snippet

```
add $t0, $s1, $s2
add $t1, $s3, $s4
sub $s0, $t0, $t1
```

Operands: Register vs. Memory

- Arithmetic instructions use register operands.
- Data transfer instructions (lw, sw) move data between memory and registers.
- **Design Principle 3: Good design demands good compromises.**

Signed vs. Unsigned

- Two's complement representation.
- Negation: Invert bits and add 1.
- Sign extension: Replicating the sign bit to preserve value when expanding bit width.

MIPS Instruction Formats

- **R-format:** op(6) | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6)
- **I-format:** op(6) | rs(5) | rt(5) | constant or address(16)

Logical Operations

- `sll` (shift left logical), `srl` (shift right logical)
- `and`, `or`, `nor`
- Useful for extracting or inserting groups of bits in a word.

Control Instructions

- `beq rs, rt, Label` (branch if equal)
- `bne rs, rt, Label` (branch if not equal)
- `j Label` (unconditional jump)

Lecture 4: Instruction Set Architecture (2)

Recap

- RISC vs. CISC
- Instructions: Arithmetic, Data Transfer, Logical, Conditional Branch.
- Basic concepts: Operands, Numeric Representation, Formats (R-format, I-format).

Today's Topic

- More control instructions
- Procedure call/return

Control Instructions: if-else

- C code:

C

```
if (i == j) f = g + h;
else f = g - h;
```

- MIPS code:

Code snippet

```
    bne $s3, $s4, Else
    add $s0, $s1, $s2
    j Exit
Else: sub $s0, $s1, $s2
Exit:
```

Loops

- `while` loop implementation using `bne` and `j`.
- Basic block: A sequence of instructions without branches (except at end) and without branch targets (except at start).

Set on Less Than

- `slt rd, rs, rt`: if ($rs < rt$) $rd = 1$; else $rd = 0$;
- `slti rt, rs, constant`
- Why not `blt`? Too complex for hardware (slower clock). MIPS uses `slt` + `bne` / `beq`.

Procedure Call (Function Call)

- **Six steps:**
 1. Place parameters in registers ($a0$ — $a3$).
 2. Transfer control to procedure (`jal`).
 3. Acquire storage for procedure.
 4. Perform procedure's operations.

5. Place result in register for caller ($v0-v1$).

6. Return to place of call (jr).

- **Instructions:**

- `jal ProcedureLabel`: Jump and Link. Saves PC+4 in \$ra (return address).
- `jr $ra`: Jump Register. Jumps to address in \$ra.

- **Stack:**

- Used to spill registers if not enough registers are available or to save local variables.
- Stack grows down (from high to low addresses).
- Stack pointer: `$sp`.

- **Nested Procedures:**

- Must save the return address (\$ra) on the stack before calling another procedure, then restore it.

Lecture 5: Instruction Set Architecture (3)

Today's Topic

- MIPS addressing
- Translating and starting a program
- A C sort example
- Other popular ISAs

MIPS Addressing Modes

1. **Immediate addressing:** `addi $s0, $s1, 5` (Operand is a constant in the instruction).
2. **Register addressing:** `add $s0, $s1, $s2` (Operand is a register).
3. **Base/Displacement addressing:** `lw $s0, 0($s1)` (Operand is at memory location `Reg + Constant`).
4. **PC-relative addressing:** `beq $s0, $s1, Label` (Address is `PC + 4 + (Constant * 4)`).
5. **Pseudo-direct addressing:** `j Label` (Address is `PC[31:28] | (Address * 4)`).

Translating and Starting a Program

- **Compiler:** C -> Assembly.
- **Assembler:** Assembly -> Machine Code (Object file).
 - Handles pseudoinstructions (e.g., `move`, `b1t`).
- **Linker:** Combines object files and libraries into an executable.
 - Resolves external references.
 - Relocates addresses.
- **Loader:** Loads executable into memory and starts execution.

A C Sort Example (Bubble Sort)

- Shows the full translation from C swap function and sort function to MIPS assembly.
- Demonstrates register saving/restoring, loops, and array access.

Other Popular ISAs

- **Intel x86 (IA-32):**
 - CISC (Complex Instruction Set Computer).
 - Evolutionary design (backward compatibility).

- Variable length instructions.
- Registers: EAX, EBX, ECX, EDX, etc.
- **ARM:**
 - RISC.
 - Most popular embedded ISA (phones, tablets).
 - Conditional execution of almost all instructions.
- **RISC-V:** Open source ISA, growing popularity.

Lecture 6: Arithmetic Operations on Integers

Outline

- Operations on integers
- Addition and subtraction
- Multiplication and division
- Dealing with overflow

Integer Addition

- Binary addition: similar to decimal, carry bits.
- Overflow: When the result cannot be represented in the given number of bits.
 - Overflow occurs if adding two positives yields a negative, or adding two negatives yields a positive.
 - MIPS `add`, `sub` cause exceptions on overflow. `addu`, `subu` do not.

Multiplication

- Start with long multiplication algorithm.
- **Hardware:**
 - Multiplicand register, Multiplier register, Product register.
 - ALU adds Multiplicand to Product if current Multiplier bit is 1.
 - Shift Multiplicand left or Product right.
- **Optimized Multiplier:**
 - Run additions in parallel or use carry-save adders.
- **MIPS Multiplication:**
 - `mult rs, rt` / `multu rs, rt`.
 - 64-bit product in `Hi` and `Lo` registers.
 - `mfhi`, `mflo` instructions to move result.

Division

- Long division algorithm.
- **Hardware:**
 - Divisor, Remainder, Quotient.
 - Subtract Divisor from Remainder; if positive, quotient bit is 1, else 0 and restore.
 - Shift right.
- **MIPS Division:**
 - `div rs, rt` / `divu rs, rt`.
 - Quotient in `Lo`, Remainder in `Hi`.

Lecture 7: Floating Point Arithmetic

Floating Point Representation

- **IEEE 754 Standard:**
 - **Single Precision (32-bit):** 1 sign bit, 8 exponent bits, 23 fraction bits.
 - **Double Precision (64-bit):** 1 sign bit, 11 exponent bits, 52 fraction bits.
- $\text{Value} = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$
- Bias: 127 for single, 1023 for double.
- Normalized numbers: leading 1 is implicit.

Floating Point Addition

1. **Align decimal points:** Shift number with smaller exponent right until exponents match.
2. **Add significands.**
3. **Normalize result:** Shift and adjust exponent. Check for overflow/underflow.
4. **Round:** Fit to available bits.

Floating Point Multiplication

1. **Add exponents:** New exponent = $\text{Exp1} + \text{Exp2} - \text{Bias}$.
2. **Multiply significands.**
3. **Normalize result.**
4. **Round.**
5. **Determine sign.**

MIPS Floating Point Instructions

- Registers: `f0—f31` (32 single precision). Can be paired for double precision.
- Operations: `add.s`, `add.d`, `sub.s`, `mul.s`, `div.s`, etc.
- Load/Store: `lwc1`, `swc1` (or `l.s`, `s.s`).
- Comparisons: `c.eq.s`, `c.lt.s`, etc., sets a condition code bit. `bc1t`, `bc1f` branch based on it.

Subword Parallelism

- SIMD (Single Instruction, Multiple Data).
- Operating on vectors of data (e.g., 4 floats at once) for graphics/audio.

Lecture 8: The Processor

Outline

- Implementation overview
- Logic design basics
- Detailed implementation for every instruction

Introduction

- **CPU Time** = Instruction Count * CPI * Clock Cycle Time.
- We will examine two MIPS implementations:

1. **Simplified Single-Cycle:** Each instruction takes one clock cycle.
2. **Pipelined:** Instructions are overlapped.

Logic Design Basics

- Combinational elements (AND, OR, ALU, MUX).
- State elements (Registers, Memory). Edge-triggered clocking.

Datapath Building

- **Instruction Fetch:** PC -> Instruction Memory -> Instruction. $PC = PC + 4$.
- **R-Format (add, sub):** Read registers -> ALU -> Write register.
- **Load/Store:** Read registers -> ALU (calculate address) -> Data Memory -> Write register (for load).
- **Branch:** Read registers -> Compare (ALU subtraction) -> AND with "Zero" output -> Update PC if equal.

Control Unit

- Decodes instruction opcode and function code.
- Generates control signals: RegDst, Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, RegWrite.
- **ALU Control:** specific logic to tell ALU what operation to do (add, sub, and, or, slt) based on ALUOp and funct field.

Single Cycle Disadvantage

- Clock cycle must be long enough for the slowest instruction (usually Load).
- Violates "Make the common case fast".
- Solution: Pipelining.

Lecture 9: The Overview of Pipeline

Recap

- Single-cycle design is functional but slow.

Pipelining Analogy

- Like a laundry assembly line (Washer -> Dryer -> Fold -> Closet).
- Overlapping tasks increases **throughput**, not individual task latency.

MIPS 5-Stage Pipeline

1. **IF:** Instruction Fetch
2. **ID:** Instruction Decode / Register File Read
3. **EX:** Execution / Address Calculation
4. **MEM:** Data Memory Access
5. **WB:** Write Back

Performance

- Ideally, $CPI = 1$.
- Speedup is roughly equal to the number of stages (under ideal conditions).
- $Speedup = (Instruction\ Count * 1 * Cycle\ Time_single) / (Instruction\ Count * 1 * Cycle\ Time_pipeline)$.

- Cycle Time_pipeline is determined by the slowest stage.

Hazards

- Situations that prevent the next instruction from executing in the next cycle.
1. **Structure Hazard:** Hardware resource conflict (e.g., single memory for both instruction and data -> solved by separate caches).
 2. **Data Hazard:** Need to wait for previous instruction to complete its data write.
 - Solution: **Forwarding (Bypassing)** - use result as soon as it is computed.
 - Load-Use Hazard: Cannot forward backward in time. Requires a **stall** (bubble).
 3. **Control Hazard:** Deciding on control action depends on previous instruction (Branch).
 - Solution: **Stall, Predict** (Assume not taken), or **Delayed Branch**.

Lecture 10-1: Instruction-Level Parallelism

Instruction-Level Parallelism (ILP)

- Parallelism among instructions.
- Pipelining is a form of ILP.
- Methods to increase ILP:
 - **Deeper Pipeline:** More stages, less work per stage, faster clock.
 - **Multiple Issue:** Start multiple instructions per clock cycle.
 - Static Multiple Issue (VLIW): Compiler groups instructions.
 - Dynamic Multiple Issue (Superscalar): Hardware schedules instructions.

Multiple Issue

- **Static Dual Issue:**
 - Packet of 2 instructions: One ALU/Branch + One Load/Store.
 - 64-bit aligned.
- **Dynamic Pipeline Scheduling:**
 - Hardware reorders instructions to avoid stalls (Out-of-Order Execution).
 - In-order issue, Out-of-order execute, In-order commit (to preserve state correctness).

Speculation

- Guess outcome of branch to execute instructions early.
- Hardware holds results in buffers until branch is confirmed.

Energy Efficiency

- High performance requires high power.
- Modern trend: Multi-core (Task-Level Parallelism) instead of just faster single cores, to manage power/thermal limits.

Fallacies and Pitfalls

- Pipelining is easy? No, hazards (especially exception handling) make it complex.
- Poor ISA design (complex addressing, etc.) makes pipelining harder.

Lecture 11: Large and Fast: Exploiting Memory Hierarchy

Chapter 5: Large and Fast: Exploiting Memory Hierarchy

Memory Hierarchy

- **The Big Picture:**
 - Ideally, we want memory to be:
 1. **Fast** (like registers)
 2. **Large** (like disks)
 3. **Cheap**
 - Reality: Fast memory is small and expensive. Large memory is slow and cheap.
 - **Goal:** Create an illusion of a memory that is as large as the disk and as fast as the cache/registers.

Memory Hierarchy Levels

- **Registers:** ~1KB, 1 cycle access.
- **L1 Cache (Data/Instruction):** ~32KB, 2 cycles.
- **L2 Cache:** ~2MB, 15 cycles.
- **Main Memory (DRAM):** ~1GB, 300 cycles.
- **Disk:** ~80GB, 10M cycles.
- *Trend:* Larger, slower, cheaper, denser as you go down.

Memory Technology

- **SRAM (Static RAM):** Fast (0.5-2.5 ns), expensive (500—1000/GiB). Used for Caches.
- **DRAM (Dynamic RAM):** Slower (50-70 ns), cheaper (10—20/GiB). Used for Main Memory.
- **Flash:** Slower, non-volatile.
- **Magnetic Disk:** Slowest (5ms - 20ms), very cheap (0.05—0.10/GiB).

Principle of Locality

- Programs access a small portion of their address space at any time.
- **Temporal Locality:** Items accessed recently are likely to be accessed again soon (e.g., instructions in a loop, induction variables).
- **Spatial Locality:** Items near those accessed recently are likely to be accessed soon (e.g., sequential instruction execution, array data).

Caching Basics

- **Block (or Line):** Unit of copying (may be multiple words).
- **Hit:** Data is found in the upper level.
 - **Hit Ratio:** Hits / Accesses.
- **Miss:** Data is not found; must retrieve from lower level.
 - **Miss Penalty:** Time to fetch block from lower level + Time to deliver to upper level.
 - **Miss Ratio:** 1 - Hit Ratio.

Direct Mapped Cache

- Location is determined by address.
- **Mapping:** `Block address % Number of blocks in cache`.

- **Address Structure:**
 - **Tag:** Bits to identify if it's the correct block.
 - **Index:** Bits to select the set/block.
 - **Offset:** Bits to find specific byte/word within the block.
- **Valid Bit:** Indicates if the data in the block is valid (1) or garbage (0).

Handling Hits and Misses

- **Read Hit:** Normal operation.
- **Read Miss:** Stall pipeline, fetch block from next level, install in cache, resume.
- **Write Hit:**
 - **Write-Through:** Update both cache and memory. (Simple, but slow write buffer needed).
 - **Write-Back:** Update only cache. Update memory when block is replaced. (Dirty bit needed).
- **Write Miss:**
 - **Write Allocate:** Fetch block to cache, then write. (Common with Write-Back).
 - **No Write Allocate:** Write directly to memory, don't fetch to cache. (Common with Write-Through).

Lecture 12: Cache Performance and Optimization

Review

- Memory hierarchy relies on locality.
- Direct mapped cache using Tag, Index, Offset.
- Write policies: Write-Through vs. Write-Back; Write Allocate vs. No Write Allocate.

Cache Performance

- **CPU Time** = (CPU execution clock cycles + Memory-stall clock cycles) * Clock cycle time.
- **Memory-stall clock cycles** = (Read-stall cycles + Write-stall cycles).
- **Read-stall cycles** = $\frac{\text{Reads}}{\text{Program}} \times \text{Read miss rate} \times \text{Read miss penalty}$
- **Write-stall cycles** = $\frac{\text{Writes}}{\text{Program}} \times \text{Write miss rate} \times \text{Write miss penalty}$ (ignoring write buffer stalls).

AMAT (Average Memory Access Time)

- $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$

Improving Cache Performance

1. Reduce Miss Rate:

- **Associativity:**
 - **Fully Associative:** Block can go anywhere. Requires searching all tags (hardware expensive).
 - **Set Associative:** Block can go in one of n places in a specific set.
 - *Benefit:* Reduces conflict misses.
 - *Cost:* Increases hit time (mux latency) and hardware complexity.
- **Block Size:** Larger blocks reduce compulsory misses (spatial locality) but increase miss penalty and may increase conflict misses.

2. Reduce Miss Penalty:

- **Multilevel Caches:** Add L2 cache. L1 is small/fast (hit time focus), L2 is larger (miss rate focus).
- Reduces the penalty seen by the processor on an L1 miss.

Sources of Misses (3 Cs)

1. **Compulsory (Cold start):** First access to a block.
2. **Capacity:** Cache cannot contain all blocks needed.
3. **Conflict (Collision):** Competition for the same set in direct-mapped or set-associative caches.

Software Optimization

- **Blocking (Tiling):** Reorganize loops (e.g., matrix multiplication) to operate on sub-matrices that fit in the cache. Maximizes accesses to loaded data before it is replaced.
- **Radix Sort vs. Quicksort:** Radix sort has fewer instructions but higher miss rate. Quicksort can be faster due to better cache behavior.

Lecture 13 (H): Dependable Memory Hierarchy & Virtual Memory

(Note: (H) likely denotes Advanced/Honors content or "Hierarchy" continuation)

Common Framework for Memory Hierarchies

- **Block Placement:**
 - Direct Mapped (1 location).
 - Set Associative (n locations).
 - Fully Associative (Any location).
- **Finding a Block:**
 - Direct/Set Associative: Index + Tag comparison.
 - Fully Associative: Search all entries (or full lookup table).
- **Replacement Policy:**
 - **LRU (Least Recently Used):** Good but complex.
 - **Random:** Easy to implement, performs surprisingly well.
- **Write Policy:** Write-through vs. Write-back.

Virtual Memory

- Use main memory as a "cache" for secondary storage (disk).
- **Goals:**
 1. Allow efficient and safe sharing of memory among multiple programs.
 2. Remove the programming burdens of small, limited main memory.
- **Address Translation:**
 - **Virtual Address:** Used by the CPU/Program.
 - **Physical Address:** Actual address in DRAM.
 - **Page Table:** Maps Virtual Page Numbers (VPN) to Physical Page Numbers (PPN).

Page Table and TLB

- **Page Table:** Stored in memory. One per process.
- **Problem:** Every memory access effectively takes 2 accesses (1 for page table, 1 for data).
- **Solution: TLB (Translation Lookaside Buffer):**
 - A small, fast cache inside the CPU specifically for address translations.
 - On TLB Hit: Physical address obtained in 1 cycle.
 - On TLB Miss: Walk the page table (slow), update TLB.

Virtual Machines (VM)

- **VMM (Virtual Machine Monitor) / Hypervisor:**
 - Maps virtual resources to physical resources (Memory, I/O, CPUs).
 - Guest OS runs in user mode. Privileged instructions trap to VMM.
- **Benefits:** Isolation, security, legacy support, cloud computing (sharing hardware).

Dependability

- **Reliability:** Measure of continuous service accomplishment (MTTF: Mean Time To Failure).
- **Availability:** Measure of service accomplishment with respect to alternation between accomplishment and interruption.
 - $Availability = \frac{MTTF}{MTTF + MTTR}$ (MTTR: Mean Time To Repair).
- **ECC (Error Correcting Code):**
 - Hamming Code: Can detect 2-bit errors and correct 1-bit errors (SEC-DED).
 - Uses parity bits.

Lecture 14: Parallel Processors from Client to Cloud

Chapter 6: Parallel Processors from Client to Cloud

Introduction

- **Goal:** Higher performance by using multiple processors (Multiprocessors, Clusters).
- Shift from single fast processor (power wall) to many efficient processors (multicore).
- **Amdahl's Law:** Limit to speedup is determined by the serial portion of the program.

Types of Parallelism

- **Instruction-Level Parallelism (ILP):** Pipelining, Multiple Issue.
- **Data-Level Parallelism (DLP):** SIMD (Single Instruction Multiple Data), Vectors, GPUs. Same operation on many data items.
- **Task-Level Parallelism (TLP):** Independent tasks running separately.
- **Request-Level Parallelism:** Handling multiple independent requests (e.g., Web server).

Parallel Hardware Classifications (Flynn's Taxonomy)

1. **SISD:** Single Instruction, Single Data (Standard Uniprocessor).
2. **SIMD:** Single Instruction, Multiple Data (Vector, GPU).
3. **MISD:** Multiple Instruction, Single Data (Rare).
4. **MIMD:** Multiple Instruction, Multiple Data (Multicore, Clusters).

Hardware Multithreading

- Allows multiple threads to share the functional units of a single processor.

- **Fine-grained:** Switch threads every cycle.
- **Coarse-grained:** Switch threads only on long stalls (e.g., L2 cache miss).
- **SMT (Simultaneous Multithreading):** Example: Intel Hyper-Threading. Instructions from multiple threads issue in the same cycle.

GPUs (Graphics Processing Units)

- Highly parallel, SIMD-oriented architectures.
- Good for graphics and data-parallel tasks (like matrix math, deep learning).
- **Structure:** Many small cores (Thread Processors) organized in lanes.
- **Programming:** CUDA, OpenCL.

Specialized Architectures (Domain Specific Architectures)

- **TPU (Tensor Processing Unit):** Google's accelerator for TensorFlow (Machine Learning).
- **NPU/DPU/BPU:** Neural/Deep-learning/Brain processing units.

Concluding Remarks

- Main content of course: Processor, Memory, Parallelism.
- Hardware thinking is important: Resource tradeoffs.
- Performance per dollar and performance per Joule drive modern design.